Operating systems

06 / Processes and concurrency

Process versus thread

A process owns an address space and resources such as open files. Threads in one process share that address space and many resources, but each thread has its own program counter, registers and stack. Sharing makes communication efficient but also requires synchronization.

User mode and kernel mode

User mode restricts access to privileged instructions and protected memory. Kernel mode lets the operating system manage devices, address spaces and scheduling. A system call enters a controlled kernel entry point, where arguments and permissions can be checked before a service is performed.

Mode switches and context switches

A mode switch changes the processor's privilege level and need not change the running thread. A context switch changes the scheduled execution context. The OS saves and restores the program counter, stack pointer and registers, and may change address-space state when switching processes.

Race conditions

A race condition occurs when a program's result depends on the timing or interleaving of concurrent operations. Two threads can both read an old counter value and then overwrite each other's increment. Protect the read-modify-write sequence with appropriate synchronization.

Original mock course material for a portfolio demo. Not an institutional handout.

Operating systems

07 / Synchronization and memory

Mutual exclusion and critical sections

Mutual exclusion allows only one participating thread at a time to execute a protected critical section. A mutex commonly enforces this rule. The lock must protect all accesses that participate in the shared-state invariant, not merely one writer.

Mutexes and semaphores

A mutex protects exclusive ownership of a critical section and is ordinarily unlocked by the thread that acquired it. A semaphore maintains a count of available permits. A counting semaphore can limit access to several identical resources, and signaling can occur from another thread.

Four deadlock conditions

Deadlock requires mutual exclusion, hold and wait, no preemption, and circular wait to hold simultaneously. A thread holds a resource while requesting another; resources cannot be forcibly taken away; and the waiting dependencies form a cycle. Preventing any one condition prevents this class of deadlock.

Page faults

A page fault traps into the OS when a virtual-memory access cannot be completed using the current mapping. The OS checks whether the access is valid. It may load a missing page, update the page table and restart the instruction; an invalid access instead causes an error. Not every page fault requires disk I/O.

FIFO, LRU and locality

FIFO evicts the page that entered memory earliest. LRU evicts the page whose most recent use was longest ago. Temporal locality means recently used data is likely to be used again; spatial locality means nearby addresses are likely to be accessed. Locality helps explain why caching pages can reduce slow memory transfers.

Original mock course material for a portfolio demo. Not an institutional handout.